

Mapúa University
School of Information Technology
Year 3 Term 1 AY 2026 - 2027

CSS142P - Modeling and Simulation Theory
Final Project Report

Priority-Tier Queueing in an LLM API Inference Service

Submitted by:

Pelayo, Agatha Fei R.
Sanchez, Jasmin Ariane C.
Santos, Lebbeus Ryan V.
Tazarte, Mico Angelo C.

Submitted to:

Mr. Martin Joseph O. Guinto

Date:

September 30, 2026

I. Executive Summary

A small LLM inference service running two GPU slots serves both free-tier (80% of traffic) and paid-tier (20%) users from a single first-come-first-served (FCFS) queue, so paying users receive no service advantage during daily peak windows. We built a discrete-event simulation in Python and SimPy and ran 30 replications of a 24-hour day for three configurations: the FCFS Baseline, a non-preemptive priority queue for paid requests (Alternative 1), and a third GPU with FCFS (Alternative 2). Under normal peak load, Alternative 1 cut average paid-tier wait by about 64% (6.24 s to 2.23 s) at the cost of a 1-second increase for free-tier users, while Alternative 2 reduced waits for both tiers to under one second. A +20% peak-traffic stress test showed the limit of the software fix: Alternative 1 still protected paid users (3.17 s), but free-tier wait rose to 24.80 s, above the Baseline's 20.51 s. We therefore recommend Alternative 1 as the immediate, no-cost intervention and a third GPU as a planned upgrade once sustained peak demand grows.

II. Problem and Decision Question

A commercial Large Language Model or LLM inference service, such as a startup, research lab, or enterprise team operating a self-managed deployment of a large language model, runs a fixed pool of 2 concurrent GPU inference slots (on a dual-GPU local host). Unlike hyperscale providers such as OpenAI or Anthropic, which expose rate limits of 500-1,000 RPM per organization (OpenAI, n.d.; Anthropic, n.d.-a, n.d.-b), smaller commercial deployments are

capacity-constrained by physical GPU count and cannot elastically scale on demand. This simulation models one such constrained deployment.

During peak usage windows, 9:00 AM to 12:00 PM and 6:00 PM to 9:00 PM, consistent with diurnal internet traffic patterns documented by Cloudflare Radar (Cloudflare, n.d.), the request arrival rate climbs from 3 to 8 requests per minute, filling both slots and growing the queue. Under the current first-come-first-served (FCFS) policy, paid-tier users (estimated at 20% of traffic volume) experience wait times comparable to free-tier users (80% of traffic), undermining the value proposition of a paid subscription.

The service operator must decide between three configurations: (1) retain the existing single FCFS queue for all requests on the 2-slot cluster (Baseline), (2) implement a non-preemptive skip-the-line priority rule for paid-tier requests on the 2-slot cluster (Alternative 1), or (3) expand hardware capacity by provisioning a 3rd GPU inference slot while maintaining the standard FCFS queue (Alternative 2).

The decision is: should the operator implement a software-based queueing policy (non-preemptive priority rule) or invest in a hardware-based scaling solution (adding a 3rd GPU) to meaningfully reduce paid-tier wait times and SLA violations without causing unacceptable degradation for free-tier users?

III. System Boundary and Conceptual Model

A. Entities

1. Incoming Application Programming Interface or API requests.

2. One request is one entity.
3. Example: a single chat-completion API call submitted by a user application.

B. Resources

1. Inference slots: 2 concurrent GPU processing units (e.g., a dual-GPU local host), each capable of serving exactly one request at a time.

C. Arrivals

1. Requests arrive continuously over 24 hours following a Poisson process.
2. The arrival rate is 3 requests per minute during off-peak hours and rises to 8 requests per minute during two peak windows: 9:00 AM – 12:00 PM and 6:00 PM - 9:00 PM.
3. Each arriving request is independently classified as paid-tier (20% probability) or free-tier (80% probability).
4. At peak load, the 2-slot cluster operates at approximately 73% utilization, producing observable queue contention.

D. Run Start and Stop

1. A run covers one 24-hour period (00:00 to 24:00).
2. The system starts empty with all 2 slots idle and no warm-up period.
3. The run ends at exactly hour 24 (86,400 seconds); requests not yet served at that point are excluded from the results.

E. Out of Scope

1. Response quality or accuracy of AI-generated outputs.
2. User abandonment or request cancellation due to queue wait.
3. Server hardware failures, slot outages, or infrastructure downtime.
4. Mid-run changes to the number of inference slots (slot count is fixed at the start of each configuration).
5. Variable request complexity (all requests are treated as homogeneous in processing cost).

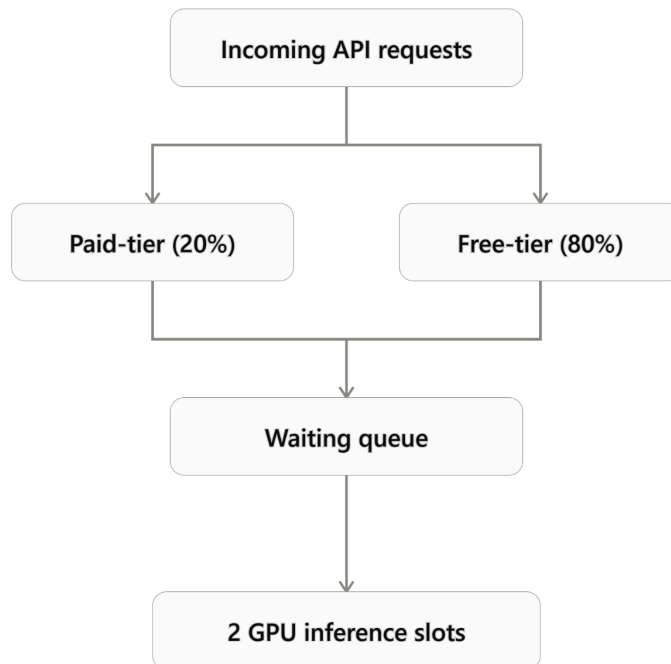


Figure 1: Conceptual Model of the API Inference Service

Figure 1 illustrates the logical flow of the baseline system architecture. Incoming API requests (the entities) enter the system and are immediately classified by user type: Free-tier (80% probability) and Paid-tier (20% probability). Under the current baseline setup, both tiers are

routed into a single, shared waiting queue. Requests remain in this queue on a strict First-Come, First-Served (FCFS) basis until one of the 2 GPU inference slots (the resources) becomes available. This visual model highlights the core bottleneck of the existing system: because the queue does not differentiate between tiers, premium Paid-tier requests are forced to wait behind Free-tier requests during peak traffic, so paying users receive no service advantage for their subscription.

IV. Input Data and Assumptions

A. Arrival rate (off-peak)

1. 3 requests/minute, modeled as a Poisson process.
2. Grounded in documented free-tier rate limits:
 - a) OpenAI free-tier legacy chat endpoints have been reported at approximately 3 RPM, while TTS-1 and Whisper-1 endpoints are currently capped at 3 RPM (OpenAI, n.d.; ScriptByAI, 2026).
 - b) The 3 req/min baseline represents the lower end of free-tier activity on a capacity-constrained deployment.

B. Arrival rate (peak)

1. 8 requests/minute, same Poisson arrival process.
2. Represents a ~2.7x surge above baseline, consistent with the pattern where paid and free users simultaneously increase activity during peak windows.

3. Informed by the observation that paid-tier accounts (OpenAI Tier 1: 500 RPM; Anthropic standard: 1,000 RPM) enable significantly higher sustained usage, producing noticeable traffic bursts on shared infrastructure (OpenAI, n.d.; Anthropic, n.d.-a, n.d.-b).

C. Peak Windows

1. 9:00 AM - 12:00 PM and 6:00 PM - 9:00 PM, based on documented diurnal internet traffic patterns.
2. Cloudflare Radar reports consistent morning and evening peaks in request volume across regions, corresponding to productivity hours and post-work usage, respectively (Cloudflare, n.d., 2024).

D. Service Time

1. Exponentially distributed with a mean of 11 seconds, derived from published LLM inference benchmarks.
2. Mid-size models (GPT-3.5-class) generate approximately 80-110 output tokens per second on standard cloud endpoints (Vellum, n.d.; Benched.ai, n.d.; Dataconomy, 2026), averaging ~ 90 tokens/sec.
3. Assuming a representative response length of $\sim 1,000$ tokens (typical for developer API completions), mean generation time is approximately $1,000 / 90 \approx 11$ seconds.
4. An exponential distribution is chosen for its memoryless property and standard use in queueing simulation.

E. Tier Split

1. 20% paid traffic / 80% free traffic, derived from freemium conversion benchmarks combined with usage amplification. Industry data places freemium free-to-paid account conversion at 2-5% on average, with a reported mean of 3.7% (First Page Sage, 2026; Artisan Strategies, 2026).
2. However, paid-tier users operate under substantially higher rate limits and are more likely to run automated or high-frequency workloads.
3. Accounting for this usage amplification effect, we conservatively estimate paid accounts generate approximately 20% of total request traffic volume, a recognized pattern in freemium API services (Userpilot, 2026).

F. Service Level Agreement (SLA) Threshold

1. 100 seconds queue wait, design parameter.
2. Defined as the boundary beyond which queue delay degrades user experience unacceptably for a synchronous API service.
3. No industry-standard SLA figure is publicly available for self-hosted LLM deployments; the value is set conservatively higher than typical interactive thresholds to reflect developer API (non-real-time) usage.

G. Inference Slots

1. Two (2), fixed design parameters representing a dual-GPU local host deployment (e.g., two A100 or equivalent cards, each serving one request at a time).
2. At a mean service time of 11 seconds, each slot processes $60 / 11 \approx 5.45$ requests per minute; two slots yield a total cluster throughput of approximately 10.9 requests per minute.
3. Under peak arrival (8 req/min), cluster utilization reaches $\rho = 8 / 10.9 \approx 73.4\%$, inducing observable queue formation and buffer delays where the non-preemptive priority rule produces measurable differences between paid-tier and free-tier wait times.
4. Under off-peak arrival (3 req/min), utilization drops to $\rho \approx 27.5\%$, reflecting realistic low-traffic conditions.
5. Alternative 2 introduces a 3rd GPU slot, expanding total cluster throughput to approximately 16.35 req/min (5.45×3), reducing peak utilization to $\rho = 8 / 16.35 \approx 48.9\%$, sufficient to observe whether hardware scaling achieves comparable wait-time reductions to the software priority rule.

V. Simulation Method and Implementation

A. Model Type

1. Discrete-event simulation (DES).
 - a) The system state, queue contents, slot occupancy, and per-request wait times change only at discrete instants: when a request arrives and when a request completes service.
2. No quantity of interest varies continuously between these events.
 - a) The event set is:
 - (1) RequestArrival
 - (2) ServiceStart, and
 - (3) ServiceCompletion.
3. Priority tier is an attribute of the entity, not a separate model type; the priority queue discipline is a standard DES scheduling mechanism.
4. DES is therefore the natural and sufficient model type for this system.

B. Implementation Tools

1. Python 3 (Google Colab runtime)
2. SimPy (discrete-event simulation framework)
3. Python's built-in random module (random variate generation); NumPy, pandas, and SciPy (statistical summarization and confidence intervals).

4. Thirty (30) independent replications per configuration (FCFS baseline on 2 GPUs, non-preemptive priority rule on 2 GPUs, and FCFS baseline on 3 GPUs), with fixed random seeds (42 to 71) recorded per replication for reproducibility.

VI. Verification and Validation

Four checks were performed to confirm that the simulation behaves as intended, and all four passed. Checks 1 to 3 verify that the code correctly implements the conceptual model. Check 4 validates the model's output against established queueing theory; all simulated averages fell within 4.0% of analytical predictions. All checks were run in the project's Google Colab notebook (see Appendix).

A. Check 1: Zero-Traffic Test (Verification)

Both the off-peak and peak arrival rates were set to 0 requests per minute for a single 24-hour Baseline run. The model should generate no requests, record no waits, and leave both GPU slots idle.

The model generated 0 requests, recorded 0 waits, and ended the run with 0 GPUs busy. The check passed. To support this test, a guard was added to the Baseline arrival generator (`arrival_generator_fcfs`) so that a zero arrival rate advances the simulation clock by 60 seconds instead of dividing by zero. This change does not affect any results at non-zero rates.

B. Check 2: Event Trace (Verification)

A single 24-hour replication of Alternative 1 (seed 42) was run with event logging enabled. Logging was added by wrapping SimPy's PriorityResource, so the model's own functions were not modified. Every arrival, service start, and completion in the run was checked automatically against three conditions:

1. No request began service while both slots were busy.
2. Under the priority rule, a waiting paid-tier request was always selected before a waiting free-tier request.
3. A request already in service was never interrupted (non-preemption).

All three conditions held across the entire run, with 0 over-capacity starts, 0 priority violations, and 0 interrupted requests. The model uses SimPy's non-preemptive PriorityResource. Table 1 shows the first 20 events of the 9:00 AM peak window, which were also inspected by hand. In it, request #1692 (paid, arrived at 32,435.88 s) began service at 32,445.28 s, ahead of request #1689 (free), which had arrived earlier at 32,414.35 s. This confirms that the priority rule operates as designed.

Table 1. Event Log Excerpt (Alternative 1, Seed 42)

Time (s)	Request	Tier	Event	Queue	Busy
32,400.26	#1687	Free	ARRIVE	0	2
32,400.26	#1687	Free	START	0	2
32,400.49	#1688	Free	ARRIVE	1	2
32,414.35	#1689	Free	ARRIVE	2	2
32,418.05	#1690	Free	ARRIVE	3	2

Time (s)	Request	Tier	Event	Queue	Busy
32,429.96	#1686	Free	DONE	3	1
32,429.96	#1688	Free	START	2	2
32,433.05	#1691	Free	ARRIVE	3	2
32,435.88	#1692	Paid	ARRIVE	4	2
32,439.89	#1693	Free	ARRIVE	5	2
32,441.92	#1694	Free	ARRIVE	6	2
32,445.28	#1688	Free	DONE	6	1
32,445.28	#1692	Paid	START	5	2
32,445.48	#1687	Free	DONE	5	1
32,445.48	#1689	Free	START	4	2
32,447.96	#1692	Paid	DONE	4	1
32,447.96	#1690	Free	START	3	2
32,449.27	#1690	Free	DONE	3	1
32,449.27	#1691	Free	START	2	2
32,449.44	#1689	Free	DONE	2	1

C. Check 3: Conservation Law (Verification)

For non-preemptive priority disciplines where all classes share the same service-time distribution, reordering the queue cannot change total delay (Kleinrock, 1976). The arrival-weighted average wait under Alternative 1 should therefore equal the Baseline FCFS wait.

1. Alternative 1: $0.2(2.23) + 0.8(7.33) = 6.31$ s
2. Baseline: $0.2(6.24) + 0.8(6.33) = 6.31$ s

The two weighted averages differ by only 0.01 s. The check passed. This confirms that the priority rule only redistributes waiting time from paid-tier to free-tier requests, without creating or removing delay.

D. Check 4: Comparison Against Queueing Theory (Validation)

Simulated average waits were compared with analytical M/M/c predictions. FCFS configurations were compared against the Erlang C formula, and Alternative 1 against the non-preemptive priority M/M/c formula (Gross et al., 2018). Analytical 24-hour averages were computed by taking the steady-state wait for each traffic window and weighting it by expected arrivals: 2,880 requests during peak windows and 3,240 during off-peak hours (3,456 peak requests in the stress test).

Table 2. Simulated vs. Analytical Average Wait Times (seconds)

Configuration	Tier	Simulated	Analytical	Difference
Baseline (2 GPUs)	Paid	6.24	6.50	-3.9%
Baseline (2 GPUs)	Free	6.33	6.50	-2.6%
Alt. 1 (Priority)	Paid	2.23	2.25	-0.8%
Alt. 1 (Priority)	Free	7.33	7.56	-3.1%
Alt. 2 (3 GPUs)	Paid	0.80	0.81	-0.5%
Alt. 2 (3 GPUs)	Free	0.82	0.81	+2.1%
Stress, Baseline	Paid	20.52	19.92	+3.0%
Stress, Alt. 1	Paid	3.17	3.17	-0.2%
Stress, Alt. 2	Paid	1.57	1.59	-1.5%

All nine simulated averages fell within 4.0% of their analytical values. The check passed. The simulation runs slightly below theory in most cases. This is expected: each 3-hour peak window starts from light off-peak conditions, so the queue needs time to build, while the analytical formulas assume the system has already reached steady state.

VII. Experiment Design

To evaluate the system, we designed three configurations, each tested under normal peak traffic and under a stress scenario:

1. **Baseline:** A 2-GPU inference cluster using a standard First-Come, First-Served (FCFS) queue.
2. **Alternative 1 (Software Scaling):** A 2-GPU inference cluster utilizing a non-preemptive priority queue, where Paid-tier requests skip ahead of Free-tier requests.
3. **Alternative 2 (Hardware Scaling):** A 3-GPU inference cluster utilizing a standard FCFS queue to test physical capacity expansion.
4. **Sensitivity Analysis (Stress Test):** All three configurations re-run with the peak arrival rate increased by 20% (8 to 9.6 requests per minute); the off-peak rate is unchanged.

For robust statistical analysis, we executed exactly 30 independent replications of each configuration under both traffic conditions. The same random seeds (42 to 71) were used for each configuration for comparability, and results are reported as means with 95% confidence intervals (t-distribution, 29 degrees of freedom). Three performance metrics were recorded: average free-tier queue wait, average paid-tier queue wait, and paid-tier SLA breach rate (the share of paid requests waiting longer than 100 seconds). All metrics are averaged over every request served in the 24-hour run, including both peak and off-peak periods.

VIII. Results and Uncertainty Analysis

Table 3. 24-Hour Average Results (Mean $\pm$ 95% CI, $n = 30$)

Metric	Baseline (2 GPUs)	Alternative 1 (Priority)	Alternative 2 (3 GPUs)
Average Free Tier Wait Time (sec)	6.33 $\pm$ 0.45	7.33 $\pm$ 0.54	0.82 $\pm$ 0.05
Average Paid Tier Wait Time (sec)	6.24 $\pm$ 0.44	2.23 $\pm$ 0.07	0.80 $\pm$ 0.06
Paid Tier SLA Breach Rate (%)	0.13 $\pm$ 0.12%	0.00%	0.00%

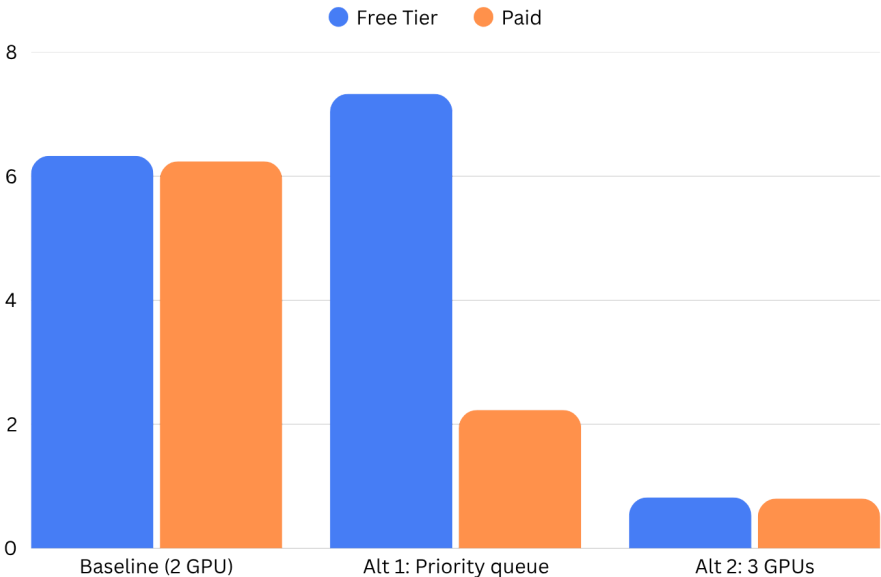


Figure 2. 24-Hour Average Wait Times by Configuration

Because these are 24-hour averages, the long off-peak periods with near-zero waits pull them down; congestion within the peak windows themselves is higher. Under the Baseline, both tiers wait about the same (6.24 s paid, 6.33 s free), which confirms that FCFS gives paying users no advantage. Paid-tier SLA breaches are already rare (0.13%) because the 100-second threshold is seldom reached at this load. Introducing the priority rule (Alt. 1) cuts paid-tier wait by

about 64%, from 6.24 s to 2.23 s, but this comes at the expense of free-tier users, whose wait rises by about 1 second, from 6.33 s to 7.33 s. As the conservation check in Section VI shows, the priority rule shifts delay from paid to free users without reducing total delay. Alt. 2 (3rd GPU) is the only option that improves both tiers at once, dropping wait times to under a second for everyone, because it removes the underlying capacity constraint rather than just reordering who waits.

Table 4. Stress Test, +20% Peak Arrival Rate (Mean $\pm$ 95% CI)

Scenario	Paid-Tier Average Wait Time (sec)	Free-Tier Average Wait Time (sec)
Baseline (2 GPUs, FCFS)	20.52 $\pm$ 2.11	20.51 $\pm$ 2.12
Alternative 1 (Priority Queue)	3.17 $\pm$ 0.10	24.80 $\pm$ 2.62
Alternative 2 (3 GPUs, FCFS)	1.57 $\pm$ 0.10	1.53 $\pm$ 0.09

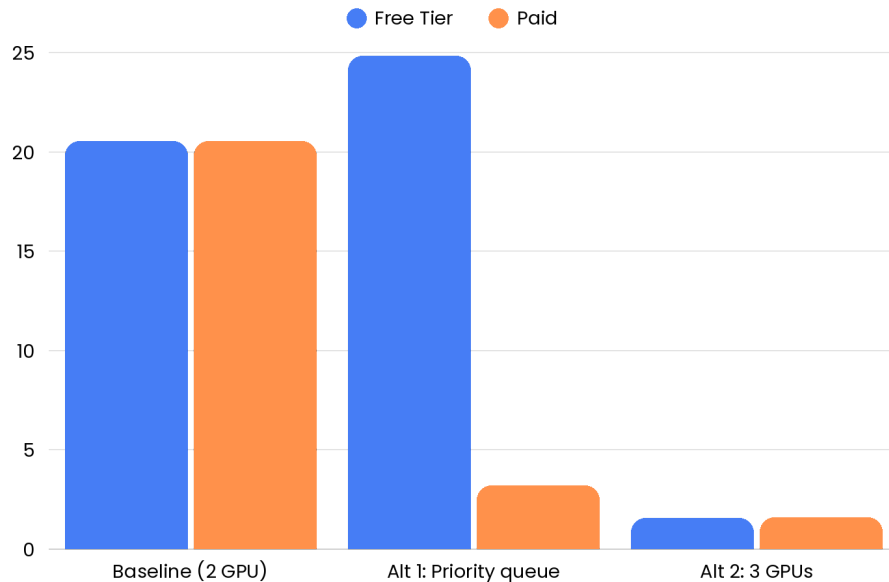


Figure 3. Wait Times Under a +20% Peak Arrival Rate

The stress test reveals how fragile the Baseline configuration really is: a 20% increase in the peak arrival rate raises peak utilization to about 88% and more than triples paid-tier wait time (6.24 s $\rightarrow$ 20.52 s), showing the current 2-GPU FCFS setup has very little tolerance for demand spikes. Both alternatives protect paid users: under Alt. 1, paid-tier wait rises only from 2.23 s to 3.17 s, and under Alt. 2 it stays under 2 seconds (0.80 s $\rightarrow$ 1.57 s).

The two alternatives differ sharply for free-tier users, however. Alt. 2 keeps free-tier wait at 1.53 s, while under Alt. 1 it rises to 24.80 s, higher than the 20.51 s free users would experience under the Baseline. This is consistent with the conservation law verified in Section VI: the arrival-weighted average wait under Alt. 1, $0.2(3.17) + 0.8(24.80) = 20.47$ s, is essentially equal to the Baseline's. Under stress, the priority rule does not reduce total delay; it moves that delay from paid-tier to free-tier users. The free-tier confidence intervals for the Baseline and Alt. 1 overlap slightly, but the direction of the change is what queueing theory predicts.

IX. Recommendation and Limitations

A. Recommendation

Based on the simulation results, we recommend implementing the software-based queueing policy (**Alternative 1: Priority Queue**) as the immediate intervention, with a third GPU (**Alternative 2**) as a planned upgrade if peak demand grows.

Alternative 1 directly addresses the stated problem. Under normal peak load, it cuts average paid-tier wait by about 64% (6.24 s to 2.23 s) at the cost of a modest free-tier increase of about 1 second (6.33 s to 7.33 s), and it requires only a change to the scheduling logic, with no new hardware. We consider this trade-off acceptable under current demand. Both alternatives bring the paid-tier SLA breach rate to 0.00%, so breach rate does not separate them; the deciding factors are the free-tier impact and cost.

The stress test shows the limit of a software-only fix. Under a 20% peak surge, Alternative 1 still protects paid users (3.17 s), but free-tier wait rises to 24.80 s, exceeding the Baseline's 20.51 s. Because the priority rule only redistributes delay without adding capacity, it cannot protect both tiers as the cluster approaches saturation. Alternative 2 outperforms Alternative 1 on every metric in both scenarios, but it requires purchasing and operating an additional data-center GPU; an NVIDIA A100 80GB, for example, costs roughly \$7,000 to \$15,000 new, plus an estimated \$500 to \$600 per year in power and cooling (Jarvislabs, 2026). We therefore recommend Alternative 2 as a second-stage investment, to be made once sustained peak traffic grows enough that free-tier degradation under Alternative 1 becomes unacceptable.

B. Limitations

1. **Homogeneous Processing:** The simulation treats all requests as having a uniform complexity with a mean service time of 11

seconds. In reality, LLM generation times vary heavily based on prompt length and output token count.

2. **No User Abandonment:** We did not model user abandonment; in a real-world scenario, users might cancel their API request if wait times exceed the 100-second SLA, which would naturally alter queue dynamics.
3. **Perfect Reliability:** Server hardware failures, slot outages, and infrastructure downtimes were excluded from the model.
4. **Exponential Service Times:** Service times were modeled as exponential for tractability. Real LLM output lengths may be more variable, which would lengthen queues beyond what the model predicts.
5. **Loose SLA Threshold:** The 100-second SLA threshold is rarely reached in any configuration (at most 0.13% of paid requests), so average wait time, rather than breach rate, is the primary basis for comparison. SLA breach rates were also not recorded in the stress test.

X. References

Anthropic. (n.d.-a). *Our approach to rate limits for the Claude API*. Claude Help Center.
<https://support.anthropic.com/en/articles/8243635-our-approach-to-api-rate-limits>

Anthropic. (n.d.-b). *Rate limits*. Claude Platform Docs.

<https://platform.claude.com/docs/en/api/rate-limits>

Artisan Strategies. (2026, July 6). *Freemium conversion rate benchmarks 2026*
(2–5% typical).

<https://www.artisangrowthstrategies.com/blog/freemium-conversion-rate-benchmarks>

Benched.ai. (n.d.). *GPT-3.5 Turbo*. <https://benched.ai/models/gpt-35-turbo>

Cloudflare. (n.d.). *Internet traffic patterns*. Cloudflare Radar.

<https://radar.cloudflare.com/display/traffic-patterns>

Cloudflare. (2024, August 13). *Introducing HTTP request traffic insights on*
Cloudflare Radar. Cloudflare Blog.

<https://blog.cloudflare.com/http-requests-on-cloudflare-radar/>

Dataconomy. (2026). *GPT-3.5 Turbo*.

<https://dataconomy.com/ai-models/gpt-35-turbo/>

First Page Sage. (2026, June 12). *SaaS freemium conversion rates: 2026 report*.

<https://firstpagesage.com/seo-blog/saas-freemium-conversion-rates/>

Gross, D., Shortle, J. F., Thompson, J. M., & Harris, C. M. (2018). *Fundamentals of queueing theory* (5th ed.). Wiley.

Jarvislabs. (2026, April 19). *NVIDIA A100 GPU price in 2026: Cost per hour, cloud pricing & specs*. <https://jarvislabs.ai/blog/a100-price>

Kleinrock, L. (1976). *Queueing systems, Volume 2: Computer applications*. Wiley.

OpenAI. (n.d.). *Rate limits*. OpenAI Platform.

<https://developers.openai.com/api/docs/guides/rate-limits/>

ScriptByAI. (2026, September 4). *OpenAI API rate limits: RPM, TPM, tiers, and 429 errors (2026)*.

<https://www.scriptbyai.com/rate-limits-openai-api/>

Userpilot. (2026, June 29). *Why freemium-to-premium conversions are flopping: What the benchmarks are teaching us*.

<https://userpilot.com/blog/freemium-to-premium/>

Vellum. (n.d.). *GPT-3.5 Turbo vs Llama 3.1 70b*.

<https://www.vellum.ai/comparison/gpt-3-5-vs-llama-3-1-70b>

XI. Appendices

Google Colab Notebook Link:

<https://drive.google.com/file/d/1VTthhYFIgN67DBM7O4AIVHWScgxS1zu9/view?usp=sharing>